МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №1
по курсу “Компьютерная графика”

Основы 3D графики

Выполнил: Н. Д. Князьков
Группа: М8О-303БВ-24
Преподаватель: В. Д. Бахарев

Москва, 2026

Условие

Цель работы: познакомиться с основами 3D-графики: построением простых 3D-объектов, проекцией на 2D-плоскость, научиться работать с матрицами перспективной и ортографической проекции и аффинными преобразованиями. Программа написана на C++ с использованием Vulkan, GLFW и ImGui на основе стартового кода.

Вариант 11: правильный икосаэдр.

Выполненные дополнительные задания (1–6):

1. Переключение матрицы проекции (ортографическая и перспективная) в интерфейсе.
2. Элементы интерфейса для изменения позиции, поворота и растяжения фигуры.
3. Движение и поворот фигуры по сложной траектории, пауза и воспроизведение анимации, изменение скорости анимации, радиуса траектории и высоты волны.
4. Изменение цвета фигуры с помощью ColorEdit в интерфейсе.
5. Процедурные цвета вершин в зависимости от их позиции в локальной системе координат; выбранный в интерфейсе цвет умножается на цвет вершин.
6. Отрисовка нескольких объектов на сцене с использованием нескольких наборов дескрипторов (VkDescriptorSet).

Метод решения

Геометрия. Правильный икосаэдр имеет 12 вершин и 20 граней — правильных треугольников. Его вершины — углы трёх взаимно перпендикулярных «золотых» прямоугольников: $(0, \pm 1, \pm\varphi)$, $(\pm 1, \pm\varphi, 0)$, $(\pm\varphi, 0, \pm 1)$, где $\varphi = (1 + \sqrt{5}) / 2$ — золотое сечение. Все вершины удалены от центра на $R = \sqrt{(1 + \varphi^2)} \approx 1{,}902$. Вершины записаны в вершинный буфер, а грани — в индексный буфер (60 индексов, по три на треугольник), поэтому каждая вершина хранится один раз, хотя входит в пять граней. Вершины каждой грани перечислены против часовой стрелки, если смотреть на грань снаружи фигуры.

Система координат такая же, как в Vulkan: ось X направлена вправо, ось Y — вниз, камера смотрит вдоль оси +Z, а глубина в каноническом объёме вида лежит в диапазоне [0, 1].

Преобразования. Используются однородные координаты (x, y, z, 1), благодаря которым перемещение тоже записывается матрицей 4×4. Вершина переводится в пространство отсечения так: p' = P · V · M · p, где M = T · Rz · Ry · Rx · S — матрица модели (сначала растяжение S, затем повороты вокруг осей X, Y, Z, затем перемещение T), V — матрица вида (сцена отодвигается от камеры на 6 единиц по оси Z), P — матрица проекции. Произведение P · V · M считается на CPU для каждого объекта и передаётся в вершинный шейдер. Матрицы хранятся по столбцам, как mat4 в GLSL.

Матрица перспективной проекции (θ — вертикальный угол обзора, a — отношение ширины окна к высоте, n и f — расстояния до ближней и дальней плоскостей отсечения):

$$\begin{vmatrix} 1 / (a \cdot tg(\theta/2)) & 0 & 0 & 0 \\ 0 & 1 / tg(\theta/2) & 0 & 0 \\ 0 & 0 & f / (f - n) & -f \cdot n / (f - n) \\ 0 & 0 & 1 & 0 \end{vmatrix}$$

Последняя строка даёт w = z, поэтому после деления на w координаты x и y уменьшаются тем сильнее, чем дальше точка от камеры, а глубина z = n переходит в 0, z = f — в 1.

Матрица ортографической проекции (w и h — ширина и высота видимой области, w = h·a):

$$\begin{vmatrix} 2 / w & 0 & 0 & 0 \\ 0 & 2 / h & 0 & 0 \\ 0 & 0 & 1 / (f - n) & -n / (f - n) \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

Она только сдвигает и растягивает параллелепипед видимой области в канонический объём, w остаётся равным 1, поэтому размер фигуры не зависит от расстояния до камеры. Тип проекции, угол обзора и размер видимой области выбираются в окне ImGui.

Анимация. Собственное время анимации t каждый кадр увеличивается на Δt · speed, только если анимация не на паузе (Δt — время между кадрами). Поэтому пауза и изменение скорости не вызывают скачков. Траектория — окружность радиуса r в плоскости XZ, вдоль которой фигура колеблется по оси Y: x = r·cos t, y = h·sin 3t, z = r·sin t. Это

смещение прибавляется к позиции из интерфейса, а к углам поворота вокруг осей Y и X прибавляются t и t/2. В перспективной проекции видно, как фигура увеличивается, приближаясь к камере.

Цвет. Цвет вершины вычисляется из её позиции в локальной системе координат: $c = p / R \cdot 0{,}5 + 0{,}5$, то есть каждая координата переводится из $[-R, R]$ в $[0, 1]$ и становится компонентой R, G или B. Цвет из ImGui::ColorEdit3 передаётся в шейдер через uniform-буфер, и в вершинном шейдере цвет вершины умножается на него. Между вершинами растеризатор интерполирует цвет с помощью барицентрических координат.

Несколько объектов (доп. задание 6). На сцене три икосаэдра. Они используют общие вершинный и индексный буферы и один графический конвейер, но у каждого объекта свои параметры (позиция, поворот, растяжение, цвет), свой uniform-буфер с матрицей и цветом и свой набор дескрипторов VkDescriptorSet, который указывает на этот буфер. Все наборы устроены одинаково, поэтому описание набора (VkDescriptorSetLayout) одно. Пул дескрипторов рассчитан на 3 набора и 3 дескриптора типа UNIFORM_BUFFER (по одному в каждом наборе). При записи команд перед каждым вызовом vkCmdDrawIndexed командой vkCmdBindDescriptorSets привязывается набор очередного объекта, поэтому шейдер получает его матрицу и цвет.

Один общий uniform-буфер для всех объектов не подходит: GPU выполняет команды позже, после отправки командного буфера в очередь, поэтому если перезаписывать один буфер между записью команд отрисовки, все объекты нарисуются с данными, записанными последними. Объекты идут по одной траектории со сдвигом по фазе $2\pi \cdot i / 3$, поэтому они равномерно распределены по окружности. Другие способы решить ту же задачу: один большой uniform-буфер, в котором наборы указывают на разные смещения (кратные minUniformBufferOffsetAlignment, на использованной видеокарте это 64 байта), динамические uniform-буферы или push-константы.

Используемые объекты Vulkan:

– VkBuffer и VmaAllocation (Vulkan Memory Allocator) — вершинный и индексный буферы и по одному uniform-буферу на объект в памяти, доступной CPU и GPU. Память отображена сразу (флаг MAPPED_BIT), данные копируются через memcpy.

– VkDescriptorSetLayout — описание набора ресурсов: uniform-буфер в binding = 0 для вершинного шейдера; VkDescriptorPool — пул на 3 набора; VkDescriptorSet — по одному набору на объект, связанному с его uniform-буфером через vkUpdateDescriptorSets.

– VkShaderModule — вершинный и фрагментный шейдеры, скомпилированные из GLSL в SPIR-V компилятором glslc.

– VkPipelineLayout и VkPipeline — графический конвейер: вершина из двух атрибутов (позиция и цвет, по три float), топология TRIANGLE_LIST, отсечение задних граней (cullMode = BACK, frontFace = COUNTER_CLOCKWISE), тест глубины (LESS), динамические viewport и scissor.

– VkCommandBuffer — в функции render() записываются команды: начало render pass с очисткой цвета и глубины, установка viewport и scissor, привязка конвейера и буферов, затем для каждого объекта привязка его набора дескрипторов и отрисовка vkCmdDrawIndexed (60 индексов).

Uniform-блок использует раскладку std140: mat4 занимает байты 0–63, vec4 цвета — байты 64–79, поэтому в структуре на C++ не нужны выравнивающие байты. Данные копируются в uniform-буферы в render(), после того как prepare() дождался завершения предыдущего кадра (VkFence). Так CPU не перезаписывает данные, которые GPU ещё читает.

Результаты

Программа работает в реальном времени, все параметры меняются в окне ImGui «Icosahedron». Слои валидации Vulkan не выводят ошибок, в том числе при изменении размера окна и при завершении программы.

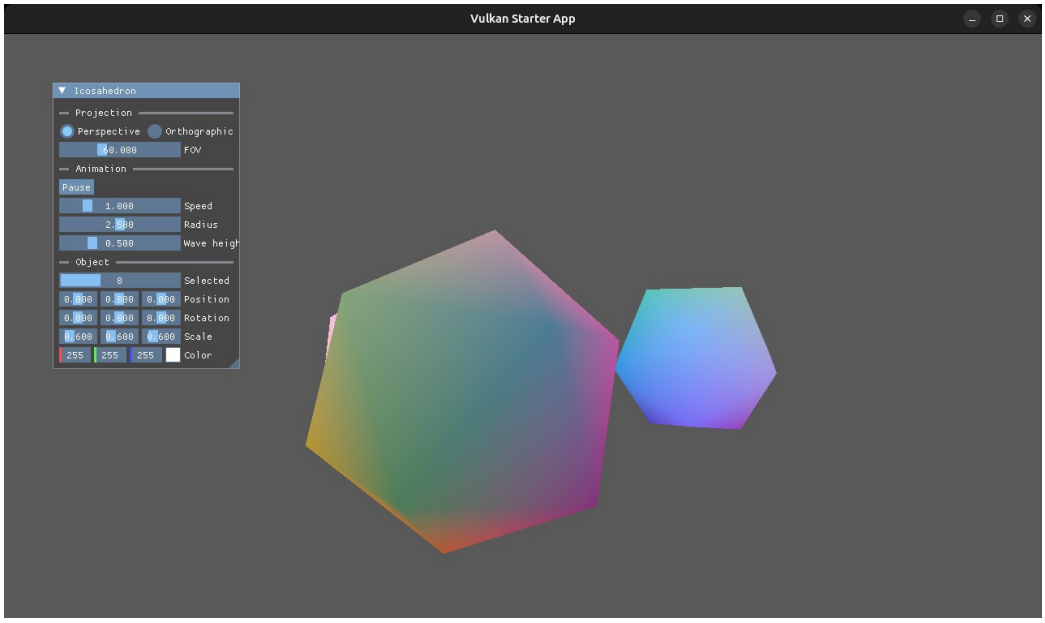


Рисунок 1 — перспективная проекция, параметры по умолчанию: три икосаэдра движутся по траектории и вращаются, ближний объект закрывает дальний (тест глубины)

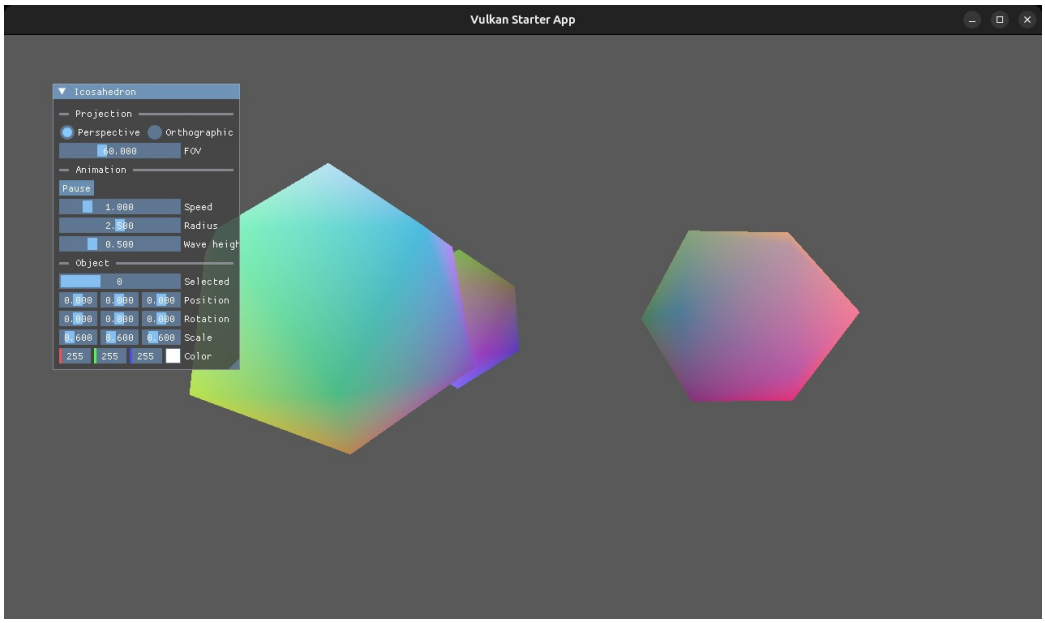


Рисунок 2 — та же сцена примерно через полторы секунды: объекты сместились по окружности, в перспективной проекции ближние объекты выглядят крупнее

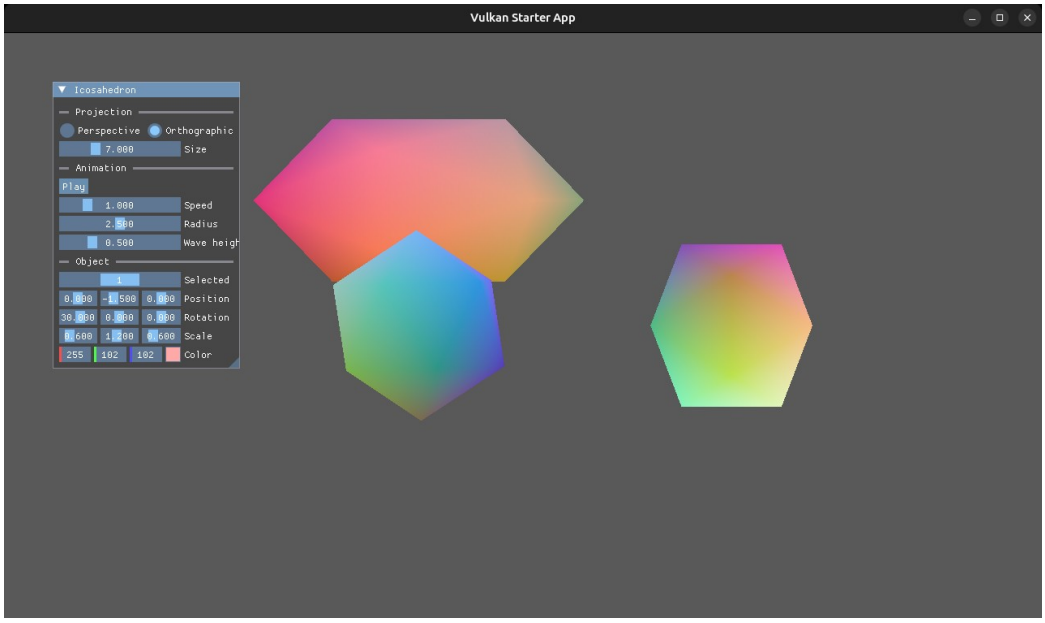


Рисунок 3 — ортографическая проекция, анимация на паузе; в интерфейсе выбран объект 1: он сдвинут вверх, повёрнут, растянут вдоль своей оси Y и имеет красный оттенок, остальные объекты при этом не изменились

Выводы

Я научился строить 3D-фигуру из вершинного и индексного буферов, передавать данные в шейдеры через uniform-буферы и наборы дескрипторов, а также настраивать графический конвейер Vulkan: вершинные атрибуты, отсечение задних граней и тест глубины. Я разобрался, как аффинные преобразования записываются матрицами в однородных координатах и почему важен порядок их умножения, чем перспективная проекция отличается от ортографической (деление на w), как нарисовать несколько объектов с разными данными, привязывая перед каждой отрисовкой свой набор дескрипторов, и как с помощью ImGui менять параметры сцены в реальном времени.